Documentação de Arquitetura e Especificação

Pipeline de Integração e Persistência de Dados - Brasileirão Série A

FIAP - Faculdade de Informática e Administração Paulista
Computational Thinking using Python - Prof. Fernando Almeida - Checkpoint 5

Integrante	RM
Diego Soares Trujillo	570147
Enzo Nukui da Silva	569770
Eduardo Damasio Guelere	569960

Repositório Git: [inserir link do repositório]

1. Visão Geral da Solução

Cenário. Resultados de campeonatos de futebol são publicados continuamente em fontes externas, mas quem precisa analisá-los (classificação, desempenho por rodada, histórico por time) depende de ter esses dados armazenados de forma estruturada e durável. A aplicação desenvolvida resolve esse problema consumindo os resultados do Campeonato Brasileiro (Série A) de uma API pública, processando-os em memória e gravando o resultado em uma tabela do banco de dados Oracle da FIAP. A partir do banco, o programa permite consultar as partidas por rodada, buscar jogos de um time e gerar a tabela de classificação de qualquer temporada.

Justificativa da API escolhida. Foi utilizada a API pública de placares (scoreboard) da ESPN, pelos seguintes motivos:

- Gratuita e sem autenticação (não exige chave de API nem cadastro), conforme pede o enunciado;
- Retorna JSON bem estruturado, com mandante, visitante, placar, data e status de cada partida;
- Permite consultar uma temporada inteira por parâmetro (`dates=ano`), o que gera massa de dados suficiente (380 partidas por temporada) para justificar o processamento e a persistência;
- Não é uma das APIs vedadas no Checkpoint (ViaCEP e PokeAPI).

Organização do código. O projeto possui dois módulos: `api.py` (consumo da API, transformações, classificação, menu e saída no terminal) e `banco.py` (conexão com o Oracle, criação da tabela, inserção e consulta).

2. Especificação da API Consumida

URL Base

```
https://site.api.espn.com/apis/site/v2/sports/soccer/bra.1/scoreboard
```

Explicação da API utilizada

Endpoint HTTP público da ESPN que devolve o placar das partidas de uma liga. O código `bra.1` identifica o Campeonato Brasileiro Série A. A requisição é feita com o método **GET** pela biblioteca `requests`, com timeout de 10 segundos.

Parâmetro	Valor usado	Função
dates	Ano da temporada (ex.: 2025)	Seleciona a temporada consultada.
limit	500	Quantidade máxima de eventos retornados (uma temporada tem 380 jogos).

Campos do JSON utilizados (dentro de `events`):

Campo	Uso no sistema
id	Identificador único da partida na ESPN (evita duplicidade no banco).
date	Data e hora da partida (formato ISO 8601).
status.type.completed / state	Define se a partida já foi encerrada.
competitions[0].competitors[].homeAway	Indica quem é mandante ("home") e visitante ("away").
competitions[0].competitors[].score	Gols marcados pelo time.
competitions[0].competitors[].team.displayName	Nome do time.

Tratamento de erros HTTP e de conexão (função `buscar_partidas`):

Situação	Tratamento
200	Converte a resposta em JSON e retorna a lista de eventos.
400	Mensagem "Consulta inválida".
403	Mensagem "Acesso negado pela ESPN".
404	Mensagem "Campeonato não encontrado".
429	Mensagem "Limite de consultas da ESPN atingido".
500 ou superior	Mensagem "Servidor da ESPN fora do ar".
Outros códigos	Exibe o código recebido.
ConnectionError / exceção genérica	Captura a falha, exibe a mensagem e retorna None, sem derrubar o programa.

3. Regras de Negócio e Transformações Aplicadas

Antes de gravar no banco, cada evento retornado pela API passa pelas transformações abaixo (função `normalizar_partida`). A aplicação executa mais de duas operações de processamento, entre filtros, cálculos, categorização e limpeza.

#	Operação	Regra
1	Filtro de partidas encerradas	Somente partidas com status concluído (completed = true ou state = "post") são aproveitadas. Jogos futuros ou em andamento são descartados.
2	Identificação de mandante e visitante	Os dois competidores são separados pelo campo homeAway. Se algum dos lados não existir, a partida é ignorada.
3	Limpeza e conversão do placar	O placar chega como texto e é convertido para inteiro. Placar ausente ou inválido descarta a partida.
4	Cálculo do resultado e dos pontos	Mais gols do mandante: VITORIA_MANDANTE (3 x 0 pontos). Mais gols do visitante: VITORIA_VISITANTE (0 x 3). Empate: EMPATE (1 x 1).
5	Cálculo de total e diferença de gols	total_gols = gols do mandante + gols do visitante. diferenca_gols = valor absoluto da diferença entre os placares.
6	Categorização por volume de gols	total_gols menor ou igual a 2: POUCOS_GOLS. Acima de 2: MUITOS_GOLS.
7	Padronização do nome dos times	Usa displayName; se não existir, usa o campo name.
8	Controle de duplicidade	Antes de inserir, o sistema consulta o ID_ESPN. Partidas já cadastradas são ignoradas, então reimportar a mesma temporada não duplica dados.

Exemplo. Para a partida Flamengo 2 x 1 Palmeiras (mandante: Flamengo), o registro gravado seria:

```
resultado       = VITORIA_MANDANTE
pontos_mandante = 3   pontos_visitante = 0
total_gols      = 3   diferenca_gols   = 1
categoria       = MUITOS_GOLS
```

Operações executadas sobre os dados gravados

- **Classificação** (`gerar_classificacao`): para cada time são acumulados jogos, vitórias, empates, derrotas, gols marcados, gols sofridos e pontos; o saldo é gols marcados menos gols sofridos. A ordenação segue os critérios: pontos, número de vitórias, saldo de gols e gols marcados.
- **Rodadas** (`agrupar_rodadas`): as partidas, ordenadas por data, são agrupadas em blocos de (número de times / 2) jogos, ou seja, 10 jogos por rodada no Brasileirão. É uma aproximação: partidas adiadas e disputadas fora de ordem podem aparecer na rodada vizinha.
- **Busca por time:** comparação sem diferenciar maiúsculas de minúsculas, em mandante ou visitante.

4. Dicionário de Dados e Script DDL (Oracle)

Tabela: **PARTIDAS_BRASILEIRAO**. Cada linha representa uma partida encerrada.

Coluna	Tipo	Descrição	Restrições
ID_PARTIDA	NUMBER	Identificador interno da partida, gerado pelo banco.	PRIMARY KEY, GENERATED ALWAYS AS IDENTITY
ID_ESPN	VARCHAR2(30)	Identificador da partida na API da ESPN.	UNIQUE, NOT NULL
ANO	NUMBER(4)	Ano da temporada.	NOT NULL
DATA_PARTIDA	VARCHAR2(35)	Data e hora da partida em formato ISO 8601, como retornado pela API.	Opcional
MANDANTE	VARCHAR2(100)	Nome do time mandante.	NOT NULL
GOLS_MANDANTE	NUMBER(2)	Gols marcados pelo mandante.	NOT NULL
VISITANTE	VARCHAR2(100)	Nome do time visitante.	NOT NULL
GOLS_VISITANTE	NUMBER(2)	Gols marcados pelo visitante.	NOT NULL
TOTAL_GOLS	NUMBER(2)	Soma dos gols da partida (campo calculado).	NOT NULL
DIFERENCA_GOLS	NUMBER(2)	Diferença absoluta de gols entre os times (campo calculado).	NOT NULL
RESULTADO	VARCHAR2(30)	VITORIA_MANDANTE, VITORIA_VISITANTE ou EMPATE.	NOT NULL
CATEGORIA	VARCHAR2(20)	POUCOS_GOLS (até 2 gols) ou MUITOS_GOLS (3 ou mais).	NOT NULL
PONTOS_MANDANTE	NUMBER(1)	Pontos do mandante na partida (0, 1 ou 3).	NOT NULL
PONTOS_VISITANTE	NUMBER(1)	Pontos do visitante na partida (0, 1 ou 3).	NOT NULL

Script DDL

```
CREATE TABLE PARTIDAS_BRASILEIRAO (
    ID_PARTIDA       NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ID_ESPN          VARCHAR2(30) UNIQUE NOT NULL,
    ANO              NUMBER(4) NOT NULL,
    DATA_PARTIDA     VARCHAR2(35),
    MANDANTE         VARCHAR2(100) NOT NULL,
    GOLS_MANDANTE    NUMBER(2) NOT NULL,
    VISITANTE        VARCHAR2(100) NOT NULL,
    GOLS_VISITANTE   NUMBER(2) NOT NULL,
    TOTAL_GOLS       NUMBER(2) NOT NULL,
    DIFERENCA_GOLS   NUMBER(2) NOT NULL,
    RESULTADO        VARCHAR2(30) NOT NULL,
    CATEGORIA        VARCHAR2(20) NOT NULL,
    PONTOS_MANDANTE  NUMBER(1) NOT NULL,
    PONTOS_VISITANTE NUMBER(1) NOT NULL
);
```

Transações: as inserções de uma importação são feitas em uma única transação. Ao final é executado `commit()`; se ocorrer qualquer erro do Oracle, é executado `rollback()` e nenhuma partida daquela importação é gravada pela metade.

5. Instruções de Execução

1. Requisitos. Python 3 instalado (com a opção "Add python.exe to PATH" marcada) e acesso à rede que alcança o servidor Oracle da FIAP.

2. Baixar o projeto. Copiar a pasta cp5_futebol (arquivos api.py e banco.py) para o computador.

3. Instalar as dependências. No terminal, dentro da pasta, executar o comando abaixo.

```
pip install requests oracledb

# No Windows, se o pip nao for reconhecido:
py -m pip install requests oracledb
```

4. Configurar o acesso ao banco. As credenciais não ficam gravadas no código. O programa lê as variáveis de ambiente `DB_USER`, `DB_PASSWORD` e (opcionalmente) `DB_DSN`. Se `DB_USER` ou `DB_PASSWORD` não estiverem definidas, o programa pede o usuário (RM) e a senha ao iniciar; a senha digitada não aparece na tela. Para definir as variáveis no Windows (no mesmo terminal em que o programa será executado):

```
set DB_USER=RMxxxxxx
set DB_PASSWORD=sua_senha
# DB_DSN e opcional (padrao: oracle.fiap.com.br:1521/ORCL)
```

5. Executar.

```
python api.py

# ou, no Windows:
py api.py
```

6. Usar o programa. Na primeira execução a tabela é criada automaticamente (se ainda não existir). O programa pede o ano da temporada, importa as partidas da ESPN para o Oracle e abre o menu:

```
1 - Navegar pelas rodadas
2 - Ver uma rodada específica
3 - Buscar partidas por time
4 - Ver classificação
5 - Trocar temporada
0 - Sair
```

Sugestão de teste: informar o ano 2025 (temporada encerrada, com as 380 partidas), navegar pelas rodadas (Enter avança, p volta, s sai), buscar um time e abrir a classificação.